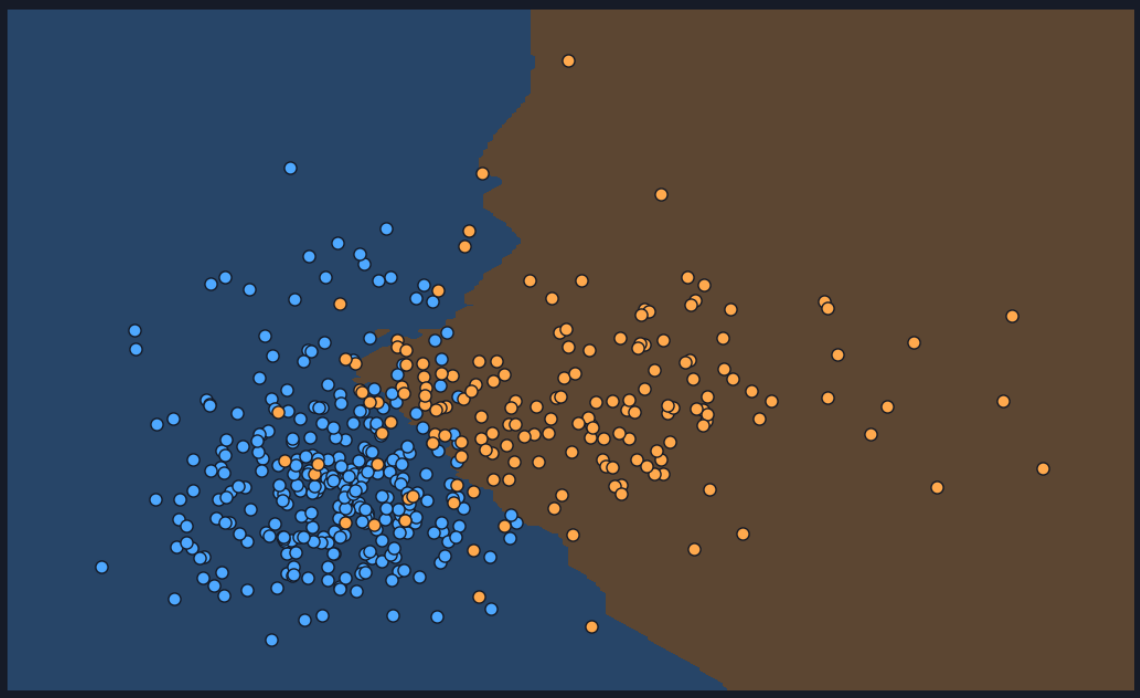
MACHINE LEARNING | SUPERVISED LEARNING | CLASSIFICATION

K-Nearest Neighbors

(KNN) from Zero to Pro

A visual, step-by-step tutorial: intuition, the mathematics, hand-worked examples, scikit-learn code, decision surfaces, over- and under-fitting, the six failure cases, and pro-level techniques.



KNN decision surface (K = 15) on two features of the breast-cancer dataset. Every coloured pixel is a prediction.

Level 1 - Beginner	Level 2 - Intermediate	Level 3 - Advanced	Level 4 - Pro
Intuition, geometry, the 5-step algorithm	Distance maths, scaling, scikit-learn, choosing K	Decision surfaces, over/under-fitting, 6 failure cases	Weighted KNN, regression, KD-trees, pipelines, from-scratch code

Expanded from the Data Science Mentorship lecture on K-Nearest Neighbors. Nothing from the lecture is skipped: every idea has been kept, re-ordered into tutorial form, and extended with figures, maths and code.

Contents

Part 0 - Welcome and Roadmap 4

Resources mentioned in the original lecture 4

Prerequisites (all easy) 4

Vocabulary you will meet 4

Part 1 - The Intuition 5

1.1 Birds of a feather 5

1.2 Our running problem: student placement 5

1.3 The whole algorithm in one sentence 6

1.4 Three words you will hear about KNN 6

Part 2 - How KNN Works, Step by Step 7

2.1 The geometric picture 7

2.2 The five steps 7

2.3 The mathematics you need 7

2.4 A complete worked example by hand 8

2.5 Pseudocode 9

2.6 Ties and the "odd K" rule 9

2.7 More than two classes, and numbers instead of classes 9

Part 3 - Distance: How KNN Measures "Close" 10

Part 4 - Coding KNN on a Real Dataset 11

4.1 The dataset 11

4.2 The six-step recipe 11

4.3 Step 2 - Train/test split 12

4.4 Step 3 - Feature scaling (very important for KNN) 12

4.5 Steps 4 and 5 - Train and predict 13

4.6 Going beyond accuracy: the confusion matrix 14

4.7 Predicting a brand-new patient 14

Part 5 - Choosing the Best K 15

5.1 Method 1 - The heuristic: square root of N 15

5.2 Method 2 - Experimentation 15

5.3 The professional upgrade: cross-validation 16

Part 6 - Decision Surfaces 17

6.1 What is a decision surface? 17

6.2 Why is it useful? 18

6.3 How is it generated? 18

6.4 The one-liner: mlxtend 19

Part 7 - Overfitting and Underfitting in KNN . . . 20

7.1 K = 1: memorising the data (overfitting) . . . 20
7.2 K = N: ignoring the data (underfitting) . . . 20
7.3 The experiment (code demo, as a picture) . . . 20
7.4 The deeper idea: bias and variance . . . 22

Part 8 - The Six Failure Cases of KNN . . . 22

8.1 Failure 1 - Large datasets (it becomes slow) . . . 22
8.2 Failure 2 - High dimensions (the curse of dimensionality) . . . 23
8.3 Failure 3 - Outliers (and a badly chosen K) . . . 24
8.4 Failure 4 - Features on different scales . . . 24
8.5 Failure 5 - Imbalanced data . . . 25
8.6 Failure 6 - Good for prediction, not for inference . . . 26
8.7 Summary of the six failure cases . . . 26

Part 9 - Pro-Level KNN . . . 27

9.1 Weighted KNN: closer neighbours deserve a louder vote . . . 27
9.2 KNN for regression . . . 28
9.3 Making KNN fast . . . 28
9.4 Fighting high dimensions . . . 29
9.5 Imbalanced data - a practical recipe . . . 30
9.6 The production pattern: Pipeline + GridSearchCV . . . 30
9.7 KNN completely from scratch (NumPy) . . . 30
9.8 Theory corner: how good can KNN get? . . . 31
9.9 Other uses of the neighbour idea . . . 31
9.10 Hyper-parameter cheat sheet . . . 31

Part 10 - Revision Kit . . . 32

10.1 One-page summary . . . 32
10.2 Should I use KNN? A quick guide . . . 32
10.3 Interview questions . . . 32
10.4 Practice exercises . . . 33
10.5 Glossary . . . 33

START HERE

Part 0 - Welcome and Roadmap

Read this first: what you will learn and how to use the tutorial.

Welcome! In the Data Science Mentorship journey we have already studied **Linear Regression** in detail. Now we move to our **second machine-learning algorithm: K-Nearest Neighbors (KNN)** - a very simple, very elegant and very lovely algorithm. Its logic matches how the real world works, which is exactly why you will not struggle to understand it.

In this single tutorial you will:

- build the **intuition** of KNN (why it works, in plain language);
- see exactly **how KNN solves a problem geometrically** and mathematically, with a worked example;
- **run the code** on a real dataset (breast cancer, 569 patients, 30 features);
- learn how to **choose K** (heuristic and experiment) and what **decision surfaces** are;
- understand how **overfitting** and **underfitting** happen in KNN;
- study the **six failure cases** of KNN (the most important skill: knowing when NOT to use an algorithm);
- finish with **pro-level** ideas: weighted KNN, KNN regression, KD-trees, pipelines, from-scratch code and interview questions.

KEY IDEA

Knowing many algorithms is good. But it is even more important to know **in which situations an algorithm works and in which it fails**. That is why Part 8 (failure cases) is the heart of this tutorial.

Resources mentioned in the original lecture

Resource	Why it helps
KNN from scratch (separate session)	Optional. If you want to code KNN without scikit-learn, this walks through it. Python knowledge is helpful but not mandatory. A from-scratch version is also included in Part 9 of this PDF.
Decision-surface video (about 40 minutes)	Shows how to draw decision surfaces by hand. You do not need to repeat it every time - the **mlxtend** library does it in one call (Part 6).
One or two additional videos in the description	Extra knowledge on related topics. Check the video description.

Prerequisites (all easy)

- What a **feature** (input column) and a **label** (output column) are.
- Mean and standard deviation (we recap them when needed).
- Basic Python: lists, NumPy arrays, pandas DataFrames (only for the code parts).

Vocabulary you will meet

Term	Meaning
Feature / input	A column that describes each example (e.g. CGPA, IQ).
Label / target / output	What we want to predict (e.g. placement: 1 = yes, 0 = no).
Class	One possible value of the label (binary problem: two classes).
Training data	Examples whose labels we already know.
Query point	A new example whose label we want to predict.
K	How many neighbours get a vote.
Neighbour	One of the training points closest to the query.
Distance	A number telling how far apart two points are.

LEVEL 1 | BEGINNER

Part 1 - The Intuition

Level 1: the idea of KNN in plain language.

1.1 Birds of a feather

The best part of KNN is that its logic is **very real-world**. Think of this famous statement by the entrepreneur and motivational speaker **Jim Rohn**:

KEY IDEA - THE KNN PRINCIPLE

"*You are the average of the five people you spend the most time with.*"

- Jim Rohn

KNN turns this sentence into an algorithm: **to know what someone is, look at the people around them**. Your neighbours tell a lot about you.

EXAMPLE

Everyday examples of the same logic

- You move to a new city and want to guess the rent of a flat: you check the rent of the 5 flats closest to it.
- You want a movie recommendation: you look at what the 10 people with the most similar taste enjoyed.
- A doctor sees a new tumour measurement: she compares it with the most similar past patients.

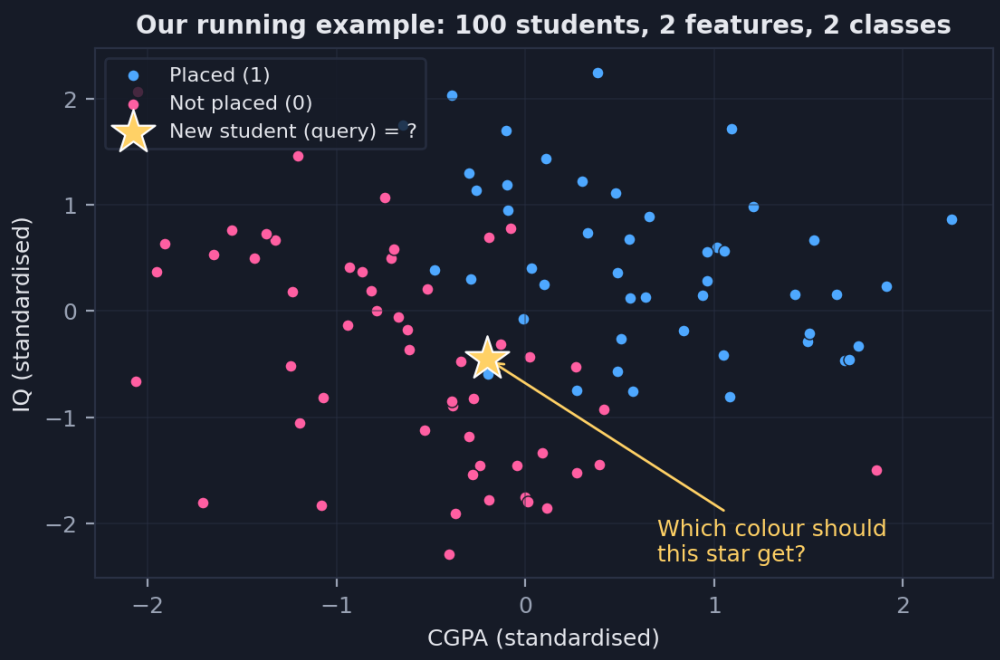
1.2 Our running problem: student placement

To explain any algorithm we use the same dataset: **CGPA**, **IQ** and whether the student's **placement happened or not**. Suppose we have data of about **100 students**. Our job is to build an ML model such that, when a **new student** arrives and we learn their CGPA and IQ, we can predict whether placement will happen.

Student	CGPA	IQ	Placement
1	8.1	118	1 (yes)
2	5.4	92	0 (no)
3	7.0	105	1 (yes)

Student	CGPA	IQ	Placement
...	...	...	...
New student	6.8	101	? (predict)

We decide to solve this with **KNN**. So how does KNN solve it?



Each dot is one training student. Blue = placement happened, pink = placement did not happen. The yellow star is the new student we want to classify.

1.3 The whole algorithm in one sentence

> **KEY IDEA**
> **To classify a new point, find the K training points closest to it and let them vote. The majority class wins.**

1.4 Three words you will hear about KNN

Word	Meaning in KNN
Supervised	We learn from examples with known labels.
Instance-based / lazy learner	There is no real training. KNN just **stores** the training data and does all the work later, when a query arrives.
Non-parametric	There is no formula with learned coefficients (unlike Linear Regression). The data itself is the model.

LEVEL 1-2 | BEGINNER TO INTERMEDIATE

Part 2 - How KNN Works, Step by Step

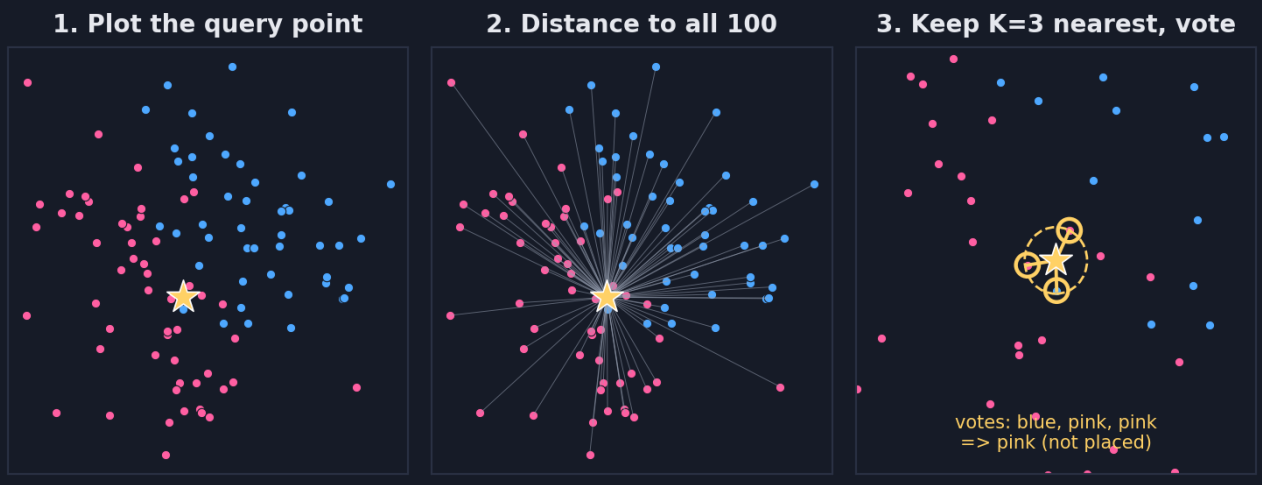
Level 1 to 2: the geometry, the algorithm, the maths and a hand-worked example.

2.1 The geometric picture

Imagine your data has **two input columns** - then every student is a **point on a 2-D plane**. We colour blue = placement happened and pink = placement did not happen. KNN works on the principle **"you are like your neighbours"**.

2.2 The five steps

Step	What you do	In our example
1	**Choose K.** Decide how many neighbours will vote.	K = 3
2	**Compute the distance** from the query point to **every** training point. (Usually Euclidean distance; other distances in Part 3.)	100 students = 100 distances
3	**Sort** all distances from smallest to largest.	closest student first
4	**Pick the K nearest** points.	the first 3
5	**Majority count**: let them vote. The class with most votes becomes the prediction.	2 blue vs 1 pink = blue



The 5 steps visualised. Note: in this example the single nearest neighbour is blue but the majority of the 3 nearest is pink - so K=1 and K=3 give different answers.

KEY IDEA

Majority count is democracy. You ask each neighbour: "tell your class". One says 1, the next says 1, the third says 0. Who is in the majority? Class 1 - so the query point is also class **1**.

You can understand it in 5 minutes: treat the query point as a **vector**, measure its distance to all other vectors, select the three closest vectors, apply the majority count and decide the class label.

2.3 The mathematics you need

Points are vectors

A student with CGPA = 7 and IQ = 105 is the vector **p = (7, 105)**. With n features, a point is a vector with n numbers: **$\mathbf{p = (p_1, p_2, \ldots, p_n)}$**.

Euclidean distance (the straight-line distance)

TWO FEATURES (Pythagoras theorem)

$$d(p, q) = \sqrt{(\text{CGPA}_p - \text{CGPA}_q)^2 + (\text{IQ}_p - \text{IQ}_q)^2}$$

ANY NUMBER OF FEATURES n

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

Read it as: subtract the points feature by feature, **square** each difference (to remove the minus sign), **add** them all, then take the **square root**.

The vote

MAJORITY COUNT

$$\hat{y} = \text{argmax}_c \sum_{i \in N_k(q)} \mathbf{1}(y_i = c)$$

$\mathbf{N_k(q)}$ is the set of the K nearest training points to query q; **$\mathbf{1(y_i = c)}$** is 1 when neighbour i has class c, else 0. So we count the votes of every class and pick the class **c** with the most votes (*argmax* = "the argument that gives the maximum").

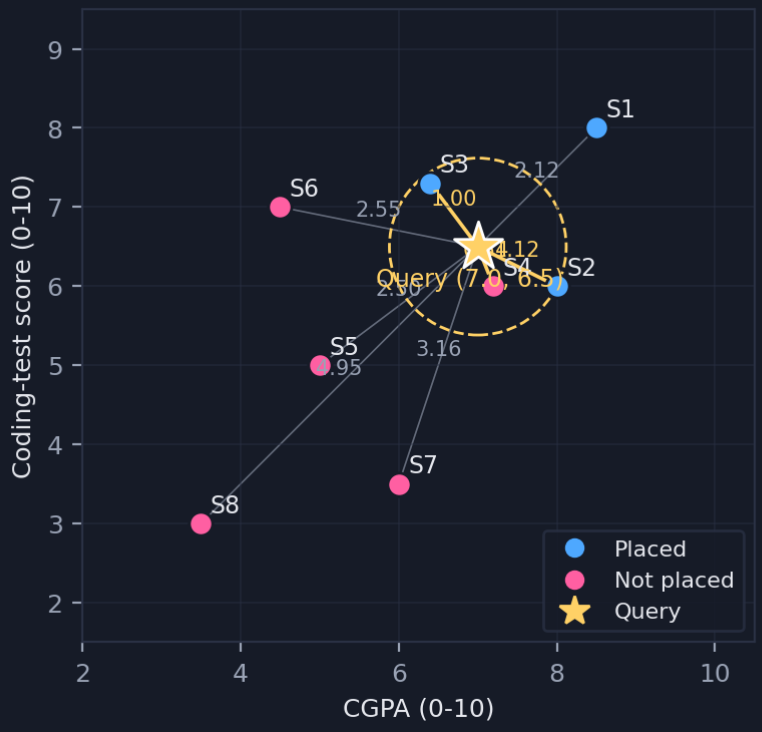
2.4 A complete worked example by hand

NOTE

For this by-hand example we use two features on the **same 0-10 scale** (CGPA and Coding-test score) so we do not need scaling yet. Scaling is a very important topic and gets its own section in Part 4.

Eight students are known. A new student has **CGPA = 7.0** and **Coding score = 6.5**. Distance to student S1 = sqrt((8.5 - 7.0)^2 + (8.0 - 6.5)^2) = sqrt(2.25 + 2.25) = **2.12**. Repeat for all students:

Student	CGPA	Coding	Class	Distance	Rank
S1	8.5	8.0	Placed	2.12	4
S2	8.0	6.0	Placed	1.12	3
S3	6.4	7.3	Placed	1.00	2
S4	7.2	6.0	Not placed	0.54	1
S5	5.0	5.0	Not placed	2.50	5
S6	4.5	7.0	Not placed	2.55	6
S7	6.0	3.5	Not placed	3.16	7
S8	3.5	3.0	Not placed	4.95	8



Distances from the query star to all eight students. The dashed circle contains the K = 3 nearest.

K	Nearest neighbours	Votes (placed : not placed)	Prediction
1	S4	0 : 1	Not placed
3	S4, S3, S2	2 : 1	Placed
5	S4, S3, S2, S1, S5	3 : 2	Placed

KEY IDEA
With K = 1 the very nearest student (not placed) decides alone. With K = 3 or 5 the crowd outvotes it. **K changes the answer**, which is why choosing K (Part 5) is the central tuning problem of KNN.

2.5 Pseudocode

Pseudocode

```
function predict(query, training_data, K):
    distances = []
    for each (point, label) in training_data:
        d = euclidean(query, point) # step 2
        distances.append((d, label))
    sort distances by d (ascending) # step 3
    nearest = first K entries # step 4
    return the most common label in nearest # step 5
```

2.6 Ties and the "odd K" rule

If K = 4 and the votes are 2 blue : 2 pink, nobody wins - a **tie**. In a two-class problem choosing an **odd K** (3, 5, 7 ...) guarantees there is never a tie, so no conflict can happen for the student. (With more than two classes ties can still occur; scikit-learn then picks the class that comes first. Weighted voting in Part 9 breaks most ties naturally.)

2.7 More than two classes, and numbers instead of classes

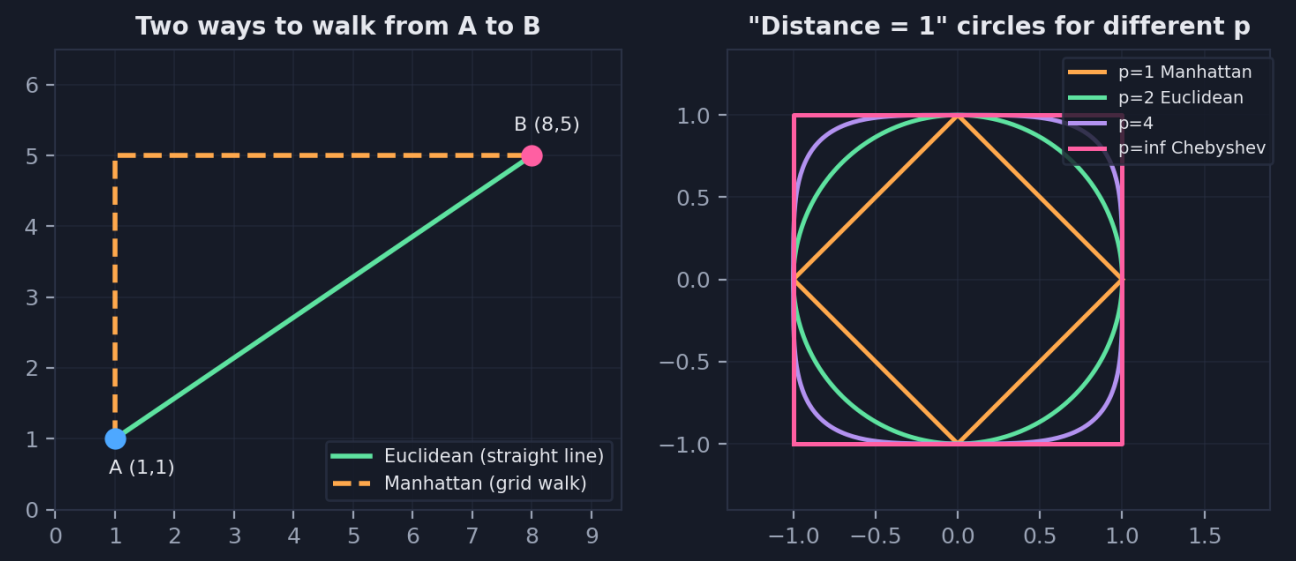
- **Multi-class**: nothing changes - the majority among K neighbours can be any of several classes (for example setosa, versicolor, virginica).
- **Regression**: instead of voting, take the **average** of the K neighbours' numeric values. See Part 9.

LEVEL 2 | INTERMEDIATE

Part 3 - Distance: How KNN Measures "Close"

Level 2: Euclidean distance is the default, but it is not the only choice.

KNN is nothing but "distance + voting". In the lecture we assumed Euclidean distance, and mentioned that in certain cases other distances are used. Here they are.



Left: two ways to travel from A to B. Right: the set of points at distance exactly 1 from the centre, for different values of p.

EUCLIDEAN (p = 2) straight-line distance

$$d_2(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

MANHATTAN (p = 1) walk along the grid like city blocks

$$d_1(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i|$$

MINKOWSKI the general family (p=1 and p=2 are special cases)

$$d_p(\mathbf{p}, \mathbf{q}) = \left(\sum_{i=1}^{n} |p_i - q_i|^p \right)^{1/p}$$

CHEBYSHEV (p = infinity) only the biggest gap matters

$$d_\infty(\mathbf{p}, \mathbf{q}) = \max_i |p_i - q_i|$$

COSINE DISTANCE compares direction, not size (text, embeddings)

$$d_{\cos}(\mathbf{p}, \mathbf{q}) = 1 - \frac{\mathbf{p} \cdot \mathbf{q}}{|\mathbf{p}||\mathbf{q}|}$$

HAMMING counts mismatching positions (categorical / binary features)

$$d_H(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} \mathbf{1}(p_i \neq q_i)$$

Distance	Use it when	scikit-learn
Euclidean	Default; features are continuous and scaled.	metric='minkowski', p=2 (default)
Manhattan	Many features / outliers; less sensitive to one large gap.	p=1 or metric='manhattan'
Chebyshev	Worst-case difference matters.	metric='chebyshev'
Cosine	Text vectors, embeddings, sparse data.	metric='cosine' (use algorithm='brute')
Hamming	Categorical or binary features.	metric='hamming'

PRO TIP
The distance is a **hyper-parameter**, just like K. If Euclidean gives poor results, try Manhattan (p = 1) - it is one parameter change and sometimes helps noticeably in high dimensions.

LEVEL 2 | INTERMEDIATE

Part 4 - Coding KNN on a Real Dataset

Level 2: breast-cancer diagnosis with scikit-learn, end to end.

4.1 The dataset

We use the **Breast Cancer Wisconsin** dataset (downloadable from the internet / Kaggle; also built into scikit-learn) to do **classification**. It has **569 patients**. The output column **diagnosis** is **M** (malignant, 212 patients) or **B** (benign, 357 patients).

The raw CSV has two columns that are useless for us: the **id** column and a column called **Unnamed: 32** that contains no values at all. We drop both. What is left: **diagnosis** (output) and **30 numeric input columns** - ten measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension) each as *mean*, *standard error* and *worst*. There are **no missing values**, so we can apply KNN directly.

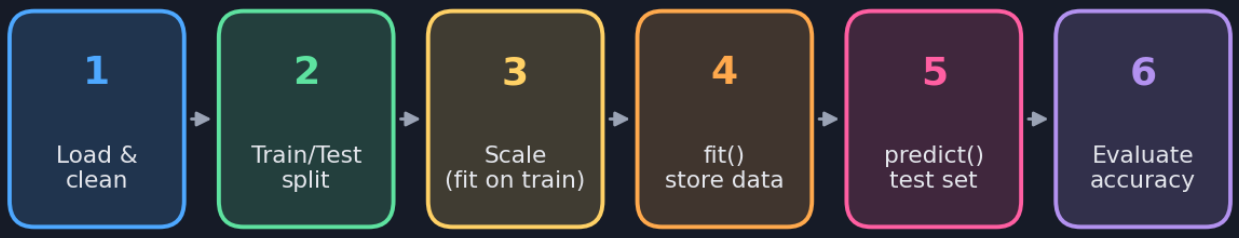
Load and clean

```
import numpy as np
import pandas as pd

df = pd.read_csv('data.csv')
df.drop(columns=['id', 'Unnamed: 32'], inplace=True) # useless columns
print(df.shape) # (569, 31) -> diagnosis + 30 features
print(df.isnull().sum().sum()) # 0 -> no missing values

X = df.iloc[:, 1:] # 30 input columns
y = df.iloc[:, 0] # diagnosis (M / B)
```

4.2 The six-step recipe



The same 6-step recipe works for almost every scikit-learn model

Almost every scikit-learn project follows this recipe.

4.3 Step 2 - Train/test split

To judge a model honestly we divide the data into **two parts**. We **train** on one part and **test** on the other, which the model has never seen. We predict the test rows, compare the predictions with the real answers, and compute a score. In Linear Regression we used the **R^2 score**; for classification we use **accuracy**.

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

With **test_size = 0.2** (20% for testing) our 569 patients become **455 for training** and **114 for testing**.

Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)
print(X_train.shape, X_test.shape) # (455, 30) (114, 30)
```

4.4 Step 3 - Feature scaling (very important for KNN)

Whenever you work with KNN you should bring all columns to the **same scale**. Look at our data: **radius_mean** is in the tens (14, 11, 19 ...) whereas **compactness_mean** is in decimals (0.08, 0.13 ...). Other columns live on yet different scales.

Column	Smallest	Largest
area_mean	144	2501
radius_mean	6.98	28.11
compactness_mean	0.019	0.345
smoothness_mean	0.053	0.163

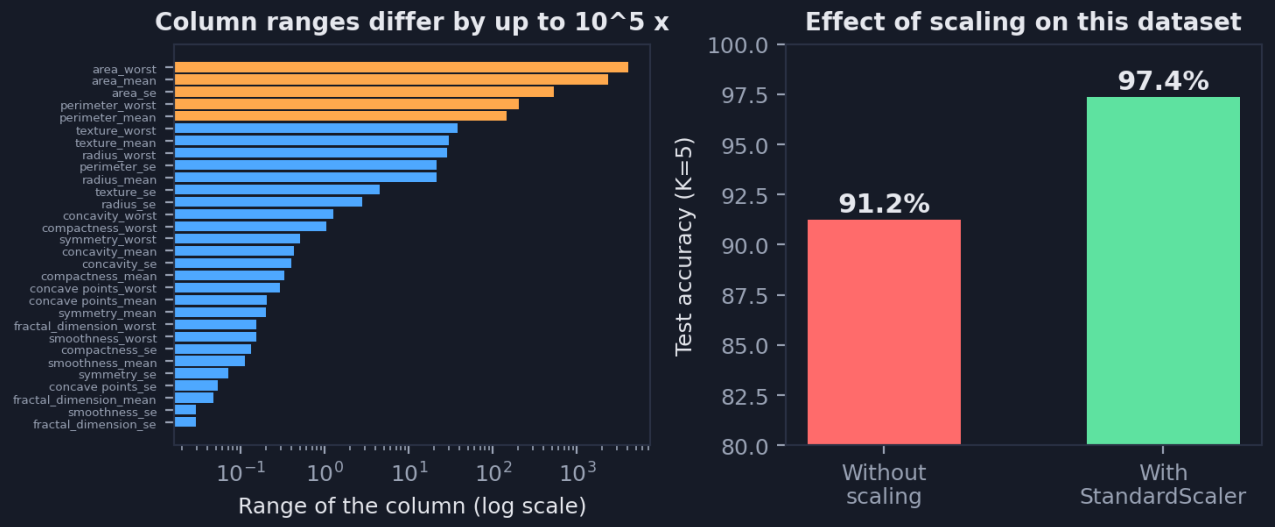
If one column is on the scale of 10,000 and another on the scale of ones, the large column swallows the distance and the small one is ignored. We want the distance to stay **reliable**, so both columns must be on the same scale. We use the **StandardScaler** transformation.

STANDARDISATION (z-score)

$$z = \frac{x - \mu}{\sigma} \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

EXAMPLE

Take a column **Age** = [20, 30, 40]. Mean = 30, standard deviation = sqrt((100+0+100)/3) = 8.16. The z-scores are (20-30)/8.16 = **-1.22**, **0**, and **+1.22**. Every column becomes centred at 0 with spread 1, so no column dominates.



Left: the 30 columns differ in range by orders of magnitude. Right: on this dataset scaling lifts K=5 accuracy from 91.2% to 97.4%.

WATCH OUT

fit on train, only transform on test. The scaler learns the mean and standard deviation from **X_train only** (*fit_transform*). On X_test we call just *transform* - we do **not** fit again, because the test data must be treated as unseen future data. Fitting on the test set leaks information (**data leakage**).

Scale

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # learn mean/std from train, then transform
X_test = scaler.transform(X_test) # reuse the SAME mean/std

print(X_train.shape) # (455, 30) -> still 455 rows, 30 columns
print(type(X_train)) # numpy.ndarray (no longer a pandas DataFrame)
```

After scaling, the data still has 455 rows and 30 columns, but it has become a **NumPy array** instead of a pandas DataFrame. All the values are now on a similar scale - exactly what we wanted.

4.5 Steps 4 and 5 - Train and predict

We create the model object and say how many neighbours to carry along - how big the neighbourhood is. We use **5**, which is also scikit-learn's default. Then we **fit** using X_train and y_train. (Remember: KNN is lazy, so *fit* only stores the data.) Finally **predict** on the test data: we get one prediction for every test patient, and *accuracy_score* compares them side by side with the true labels, counting mistakes and correct answers.

Fit, predict, score

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=5) # 5 is the default
knn.fit(X_train, y_train) # just stores the training data

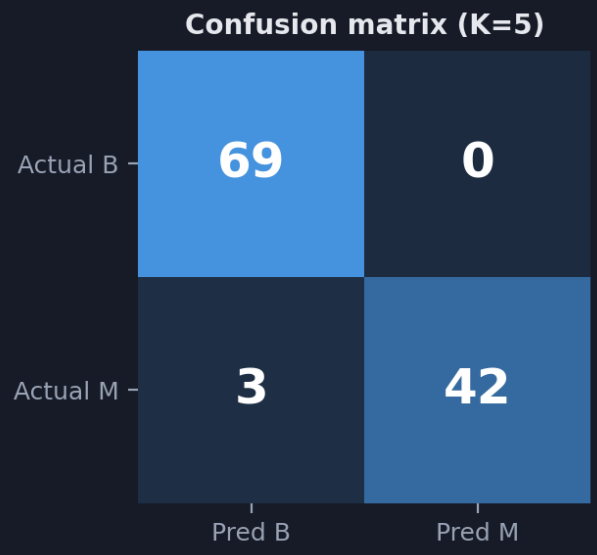
y_pred = knn.predict(X_test) # 114 predictions
print(accuracy_score(y_test, y_pred)) # 0.9737 (97.37%)
```

KEY IDEA

With K = 5 we get **97.37%** accuracy (about 97%, exactly as in the lecture). Out of 114 test patients only **3** were predicted wrongly.

4.6 Going beyond accuracy: the confusion matrix

Accuracy hides *which* mistakes happen. The **confusion matrix** shows them. In cancer detection, calling a malignant tumour "benign" is far worse than the opposite.



Rows = truth, columns = prediction. 69 benign and 42 malignant were correct; 3 malignant patients were wrongly predicted benign; 0 benign were wrongly predicted malignant.

TP = true positives, FP = false positives, FN = false negatives

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Treating **M** as the positive class: precision = 42/(42+0) = **100%**, recall = 42/(42+3) = **93.3%**. Recall is the number to watch here - it says how many real cancers we catch.

4.7 Predicting a brand-new patient

Single prediction

```
new_patient = X.iloc[[0]] # one row, 30 features (keep 2-D!)
new_scaled = scaler.transform(new_patient) # SAME scaler as training
print(knn.predict(new_scaled)) # e.g. ['M']
print(knn.predict_proba(new_scaled)) # e.g. [[0. 1.]] -> vote share of each class
```

PRO TIP

predict_proba returns the share of the K neighbours that voted for each class (with K = 5: 0, 0.2, 0.4, 0.6, 0.8 or 1). It is a rough confidence - a 5-0 vote is more convincing than 3-2.

LEVEL 2-3 | INTERMEDIATE TO ADVANCED

Part 5 - Choosing the Best K

Level 2 to 3: the biggest tuning decision in KNN.

You must be wondering: **what value of K should I use?** There is no universal answer - KNN works differently well on different datasets. To find the best K for a particular dataset there are two methods: an educated hack (heuristic) and experimentation.

5.1 Method 1 - The heuristic: square root of N

RULE OF THUMB (N = number of training observations)

$$K \approx \sqrt{N}$$

Here N = 455 training patients, so K is about sqrt(455) = **21.3**. We round to an **odd** number (21) so that votes can never tie. Treat this only as a **starting point** - researchers use it as an educated guess, not a guarantee.

5.2 Method 2 - Experimentation

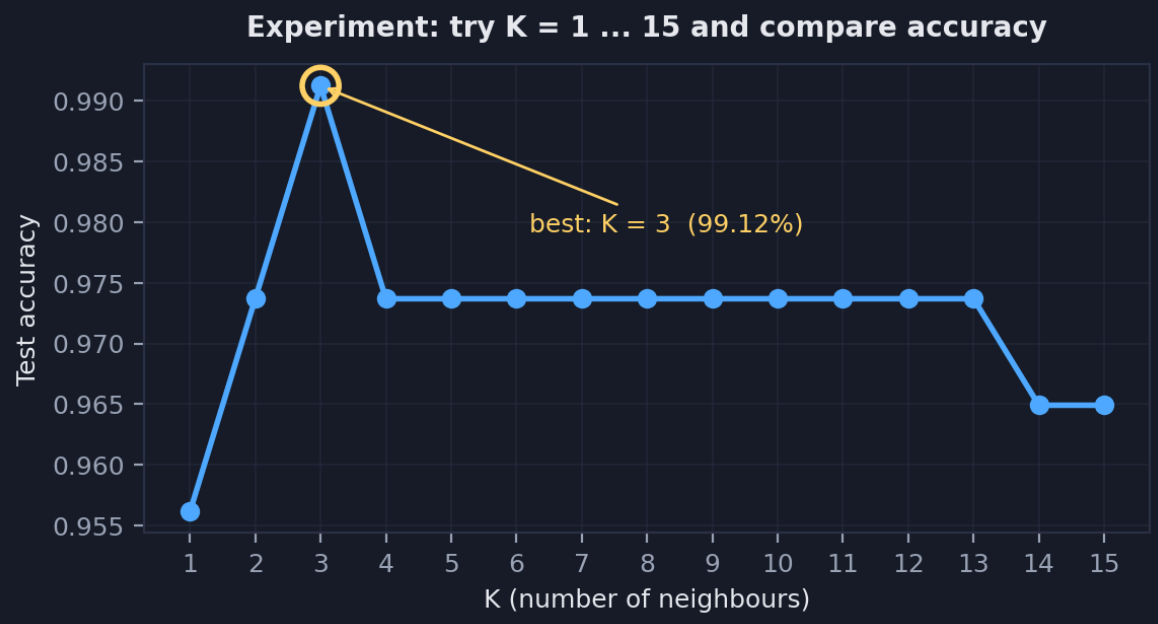
Train a different KNN model for each K (K = 1, 2, 3 ... 15), predict the test data with each, compute the accuracy score for each, and plot K on the x-axis against accuracy on the y-axis. After the loop you have 15 models and 15 accuracy scores.

Try K = 1 ... 15

```
import matplotlib.pyplot as plt

accuracy = []
for i in range(1, 16):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy.append(accuracy_score(y_test, y_pred))

plt.plot(range(1, 16), accuracy)
plt.xlabel('K'); plt.ylabel('Accuracy'); plt.show()
```



Test accuracy for K = 1 to 15. With K = 5 we got 97.4%; the best value is K = 3 with 99.12%. Smaller or larger K is worse.

The best accuracy is at **K = 3**; for K above 13 it drops and for K below 3 it drops as well. So we go back, set **n_neighbors = 3** and rebuild the model.

5.3 The professional upgrade: cross-validation

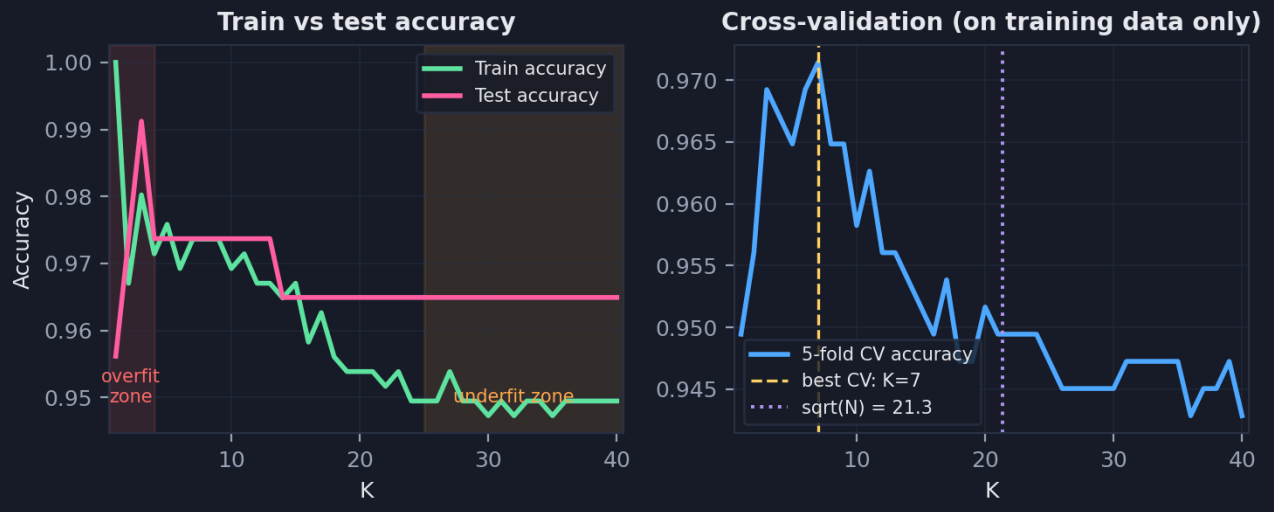
WATCH OUT
The loop above picks K by looking at the **test set**. That is slightly optimistic: the test set was supposed to stay unseen. The proper way is **k-fold cross-validation** on the **training data**, keeping the test set for one final check.

k-fold CV splits the training data into (say) 5 equal folds. Five times, it trains on 4 folds and validates on the remaining one; the 5 scores are averaged. Do this for each candidate K and choose the best average.

GridSearchCV

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(KNeighborsClassifier(),
                    param_grid={'n_neighbors': range(1, 41)},
                    cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
best = grid.best_estimator_
print(best.score(X_test, y_test)) # final, honest test score
```



Left: train accuracy (green) is perfect at K = 1 but test accuracy (pink) is not - overfitting. Right: 5-fold CV picks K = 7 (97.1%); the dotted line marks sqrt(N) = 21.3, which scores only about 94.9% - lower than K = 7.

Cross-validation chooses **K = 7**, and the final test accuracy with that K is **97.37%**. Notice that the sqrt(N) heuristic (K = 21) scores about 94.9% in cross-validation - lower than K = 7. That is exactly why the heuristic is only a **starting point** and experimentation gets the last word. Also notice how flat the curve is between K = 3 and K = 15: many K values are about equally good, which is a healthy sign.

PRO TIP

Rules to remember: (1) choose an **odd K** for two classes; (2) very small K = noisy, very large K = too simple (Part 7); (3) always scale first; (4) pick K by **cross-validation**, not by staring at the test score.

LEVEL 3 | ADVANCED

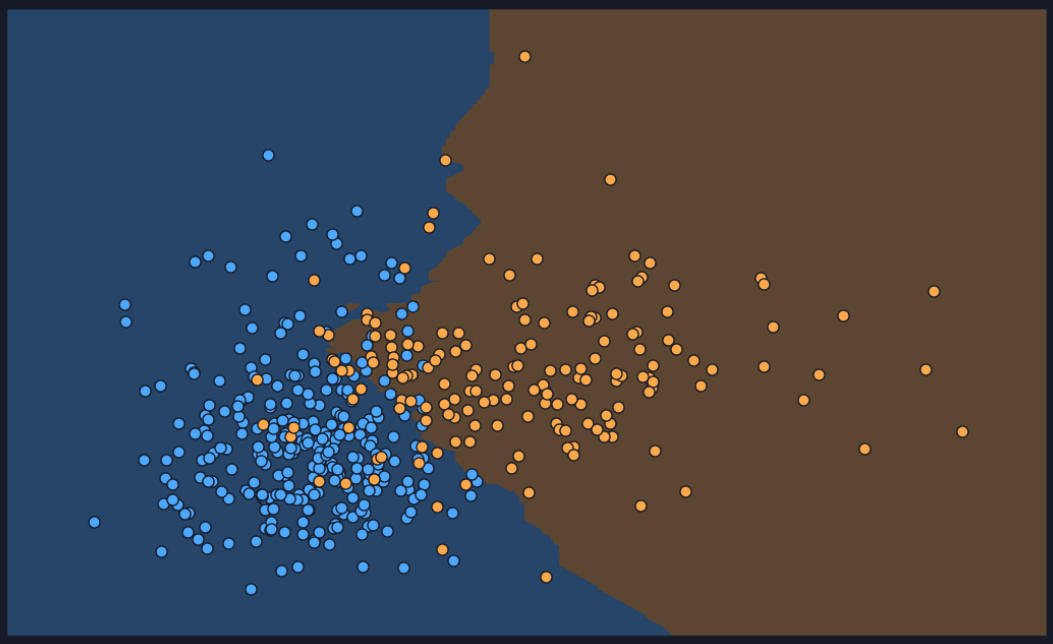
Part 6 - Decision Surfaces

Level 3: a visual tool to see what a classifier has learned.

We have seen how KNN works. Now a very interesting tool: the **decision surface** (also called the **decision boundary**). You will use it for KNN now, and again when you study **SVM, Logistic Regression, Decision Trees** and even **Neural Networks**.

6.1 What is a decision surface?

Take 2-D input data like CGPA and IQ, with a binary output (two classes, 0 and 1). The red and blue points are the **training points**. The coloured regions behind them are the **decision regions**: the whole coordinate system is divided into two regions (say orange and blue). The line where the colour changes is the **decision boundary**.



A decision surface: coloured regions = what the model would predict for a point landing there; dots = training data.

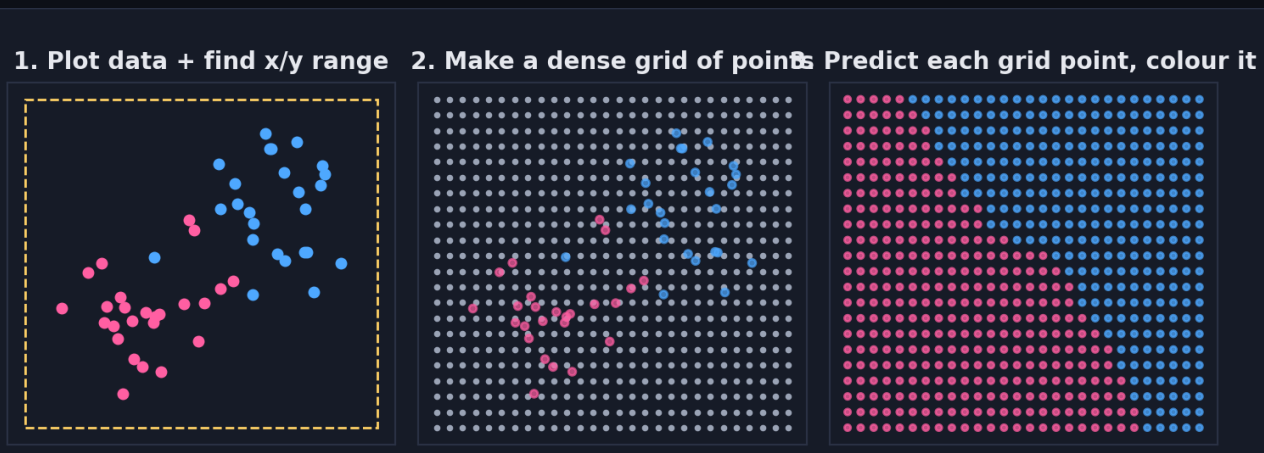
6.2 Why is it useful?

Tomorrow a new student arrives. We plot their CGPA and IQ directly on the decision surface. If the point lands in the **orange region**, the model predicts the class of the red points; if it lands in the **blue region**, the blue class. **You do not need to run the predict function separately** - you can read the answer just by looking at which region the point is in, and see where the prediction flips (the boundary). It also lets you see **how simple or complex** the model is.

> **NOTE**
> Decision surfaces can be drawn only for **1-D, 2-D or 3-D** inputs. With more features, pick two (as we do with the breast-cancer data) or compress with PCA first.

6.3 How is it generated?

- **Plot** your training data.
- Find the **range** of the data on the X and Y axes.
- Generate **lots of points** (a dense grid) covering that whole region.
- Train the KNN model on the training data and **predict every grid point**.
- **Colour-code** each grid point: if KNN says 1, colour it blue; if 0, colour it orange/pink.



The three steps. A finer grid gives a smoother picture.

Decision surface from scratch

```
# By hand with NumPy (what the 40-minute video does)
import numpy as np, matplotlib.pyplot as plt

a, b = np.arange(X[:,0].min()-.5, X[:,0].max()+.5, 0.02), np.arange(X[:,1].min()-.5, X[:,1].max()+.5,
0.02)
XX, YY = np.meshgrid(a, b) # the dense grid
grid = np.c_[XX.ravel(), YY.ravel()] # (n_points, 2)
Z = knn.predict(grid).reshape(XX.shape) # predict every grid point

plt.contourf(XX, YY, Z, alpha=0.3) # colour the regions
plt.scatter(X[:,0], X[:,1], c=y_numeric) # draw the training points
plt.show()
```

6.4 The one-liner: mlxtend

You do not need to spend 40 minutes every time. The library **mlxtend** draws decision surfaces with one function call.

mlxtend

```
# pip install mlxtend
from mlxtend.plotting import plot_decision_regions

knn = KNeighborsClassifier(n_neighbors=15).fit(X2, y_numeric) # X2 has exactly 2 columns
plot_decision_regions(X2, y_numeric, clf=knn)
plt.show()
```

KEY IDEA

Going forward, whenever you solve a classification problem with 1, 2 or 3 inputs, draw the decision surface. It is the fastest way to **see** over-fitting and under-fitting - which is our next topic.

LEVEL 3 | ADVANCED

Part 7 - Overfitting and Underfitting in KNN

Level 3: what happens as K goes from 1 to N.

Now we use decision surfaces to see how the model behaves when we increase or decrease **K**. Take the same placement data (two classes: placement happened / did not happen).

7.1 K = 1: memorising the data (overfitting)

With K = 1 the query point simply copies the class of its **single nearest neighbour**. Zoom in and you will notice that this can go wrong: a point lying in the middle of a pink area may get blue because one stray blue point happens to be closest. The decision surface breaks into **very small islands** - a little pink region here, a tiny blue region there, because around every single point a small region is carved out. The model is **memorising (mugging up) even the minor changes in the data**.

KEY IDEA

High variance means giving different results on different datasets. A K = 1 surface will not perform the same on any new dataset - its performance changes with every sample. That is **overfitting**: great on training data, worse on new data.

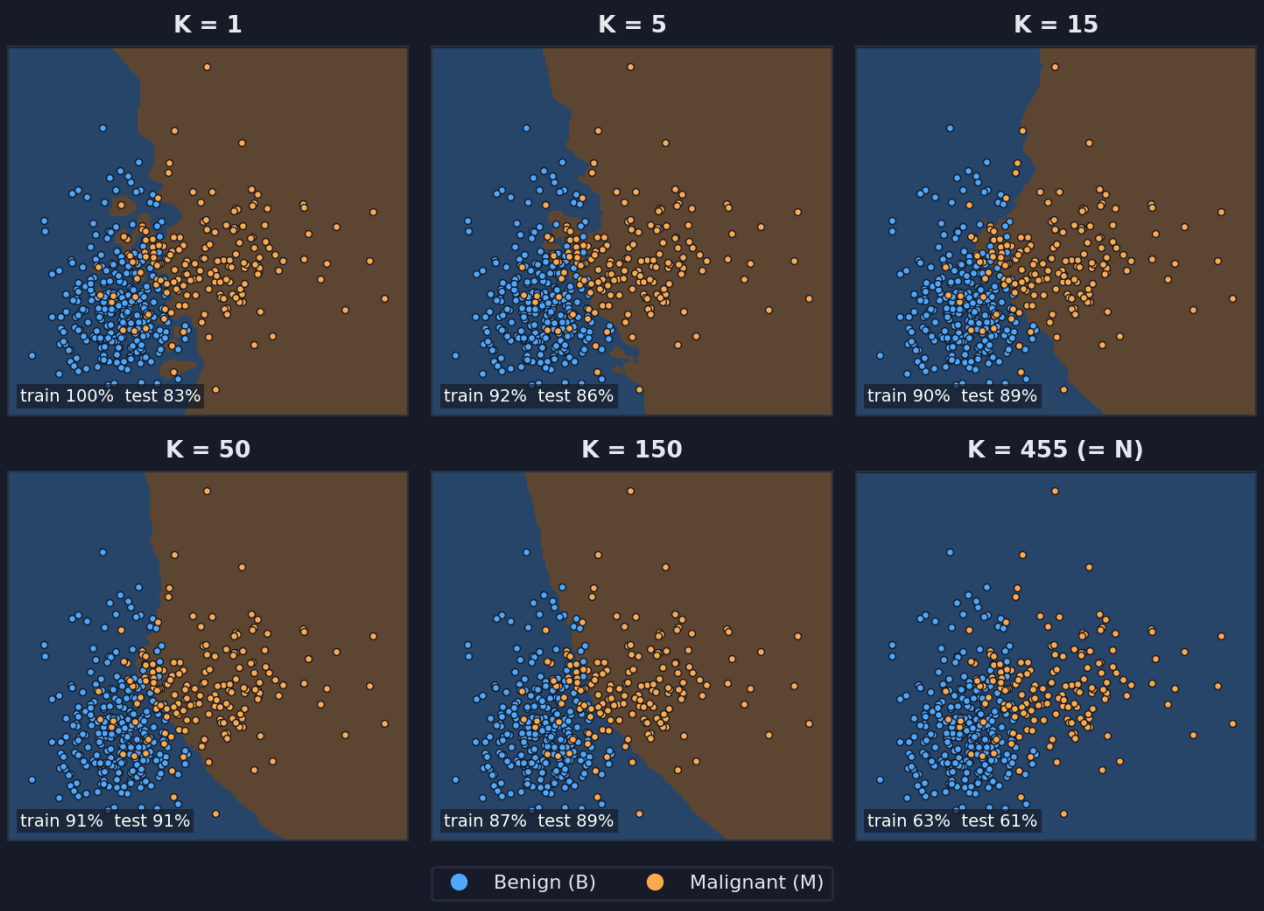
7.2 K = N: ignoring the data (underfitting)

Now the opposite. Question: **what is the maximum value of K?** It is exactly **N**, the number of training points. If N = 200 then K can be at most 200. Take K = 200: every query is compared with **all** points, and the class that dominates the dataset wins the majority count **every single time**. If blue is the majority class, then no matter where you place the query - here, here, anywhere - the answer is always blue.

This is an extremely simplistic model that is not thinking at all; it has not understood even a bit of the pattern inside the data and just gives out the majority class. That is **underfitting** (high **bias**).

7.3 The experiment (code demo, as a picture)

In the lecture a code demo with a **slider** was used: take only the **first two columns** of the breast-cancer dataset (30 columns cannot be visualised), colour the classes blue and orange, and move the K slider. Here is the same experiment as one picture:



Decision surfaces for K = 1, 5, 15, 50, 150 and K = N = 455 on two features. K = 1: train accuracy 100% but test 83% (memorising). K = N: one colour everywhere, only 61% test accuracy (underfitting).

K	What you see	Behaviour
1	Tiny islands, jagged boundary, a region around every point	Overfitting (high variance)
5	Many small bumps remain	Still somewhat noisy
15 to 50	Boundary becomes smooth; the individual pockets disappear. It is smooth and still non-linear, so it truly represents the data	Good fit
150	Boundary too straight, loses detail	Starting to underfit
N (455)	One colour everywhere	Underfitting (high bias)

TRY IT

Your assignment. Run the slider code below for different K values, observe the decision surface and **write down your observations** somewhere (a notebook is perfect). Notice at what K the small islands vanish and at what K the boundary starts to flatten.

Slider demo (Jupyter)

```
# pip install mlxtend ipywidgets
import ipywidgets as widgets
from IPython.display import display
from mlxtend.plotting import plot_decision_regions

X2 = X_train_scaled[:, :2] # first two columns only
y_num = (y_train == 'M').astype(int)

def show(k):
    model = KNeighborsClassifier(n_neighbors=k).fit(X2, y_num)
```

```
    plot_decision_regions(X2, y_num, clf=model)
    plt.title(f'K = {k}'); plt.show()

widgets.interact(show, k=widgets.IntSlider(min=1, max=100, step=1, value=5))
```

7.4 The deeper idea: bias and variance

THE "EFFECTIVE NUMBER OF PARAMETERS" OF KNN

$$\text{model complexity} \approx \frac{N}{K}$$

With K = 1 the complexity is N (one "parameter" per point - maximum flexibility, overfit). With K = N it is 1 (a single constant answer - underfit). So **K controls the complexity of KNN**, exactly like the polynomial degree controlled complexity in regression. The train-vs-test curve in Part 5 (left panel) shows the classic U-shape of the test error.

KEY IDEA

Summary. A very small K leads to **overfitting**; a very large K leads to **underfitting**. You must find a value in the middle, and you can only discover it through **experimentation** (cross-validation).

LEVEL 3-4 | ADVANCED TO PRO

Part 8 - The Six Failure Cases of KNN

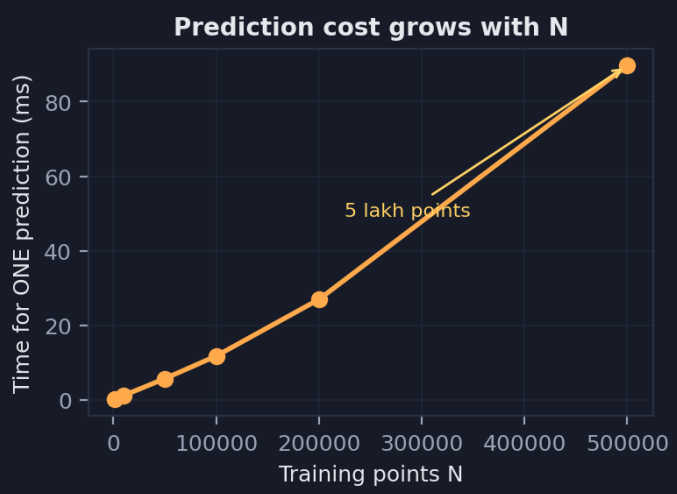
Level 3 to 4: knowing when NOT to use KNN.

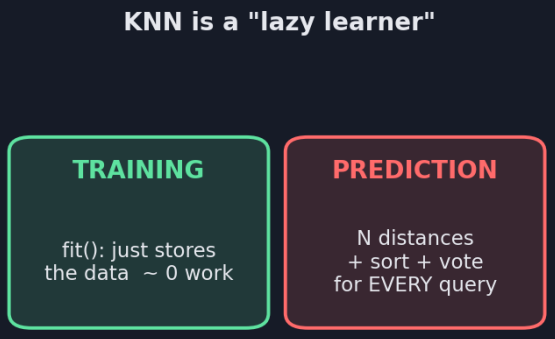
There are many algorithms and you will learn to use them. But even more important is the ability to understand **in which situation a machine-learning algorithm can be used and in which it cannot**. Here are the six places where KNN fails, each with a picture, the reason, and a fix.

8.1 Failure 1 - Large datasets (it becomes slow)

Suppose the dataset has **5 lakh (500,000) observations**. KNN does **all** its work at the moment a query arrives - the prediction phase: calculate distances, sort them, apply majority count. In training it actually does **nothing**; it just stores the points. This is why KNN is called a **lazy learner**.

When a new point comes, you calculate its distance to how many points? To all 5 lakh - that is **5 lakh distances**, then you sort 5 lakh numbers, then apply the majority count, all at prediction time. Imagine a user fills a form on a website and clicks **Predict**: only then does all this work start. Suppose it takes **3 seconds**. For internet companies, where **low latency** matters a lot, a 3-second wait means the customer leaves your platform. This is the **biggest drawback of KNN**: on small datasets results come quickly and well, but on very large datasets prediction takes too long.





Measured on our machine with fully vectorised NumPy: one prediction against 500,000 points x 30 features takes about 90 ms, and time grows in direct proportion to N. A naive Python loop would be much slower, which is closer to the scenario in the lecture.

N = training points, d = features

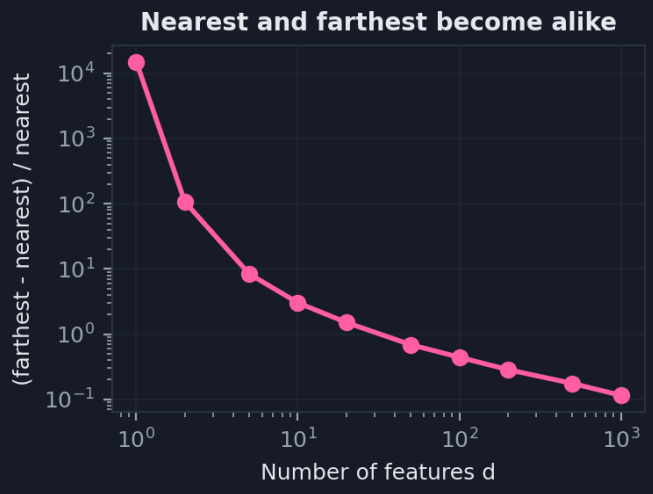
$$\text{cost per prediction} = O(N \cdot d) \; + \; O(N \log N) \text{ (sorting)}$$

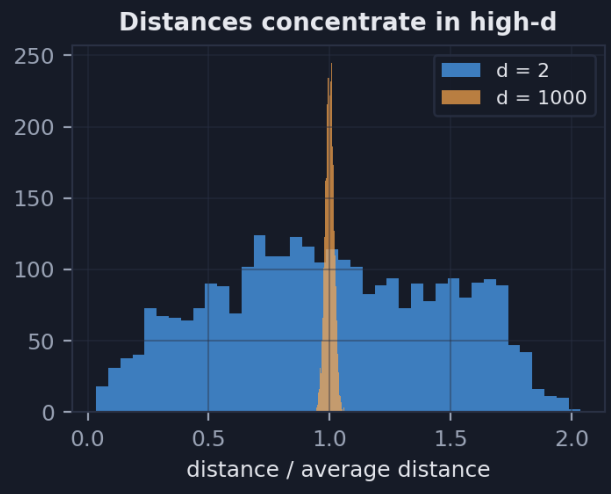
PRO TIP

Fixes: use a smarter search structure (KD-tree, Ball-tree), use approximate nearest-neighbour libraries (FAISS, Annoy, HNSW), reduce N by sampling or prototypes, reduce d with PCA, and use **argpartition** instead of a full sort. See Part 9.

8.2 Failure 2 - High dimensions (the curse of dimensionality)

With 10 features KNN is fine. With **500 features** the concept of distance itself stops being reliable. In high dimensions Euclidean distance cannot properly tell near from far. Since KNN relies **completely** on distance, if distance cannot be calculated properly, KNN's results cannot be reliable either.





We drew 1,000 random points and measured how much farther the farthest point is than the nearest. For d = 2 the ratio is about 107; for d = 1000 it is only 0.11. Right: in 1000-D almost every point is at about the same distance.

TO CAPTURE ONLY 1% OF THE DATA...

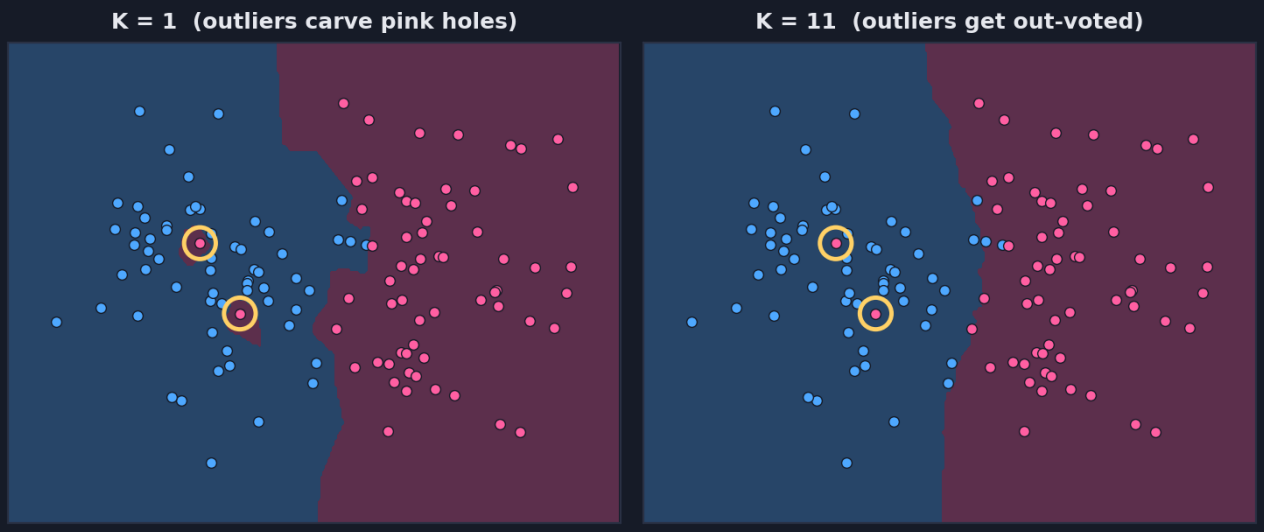
$$\text{edge length needed} = r^{1/d} \quad \Rightarrow \quad 0.01^{1/10} \approx 0.63, \quad 0.01^{1/100} \approx 0.95$$

The formula shows why: to capture just 1% of uniformly spread data in 10 dimensions your "neighbourhood" must already span 63% of every axis - it is no longer *local*. In 100 dimensions it spans 95%. "Nearest neighbours" are not near any more.

PRO TIP
Fixes: feature selection, PCA / other dimensionality reduction, domain-specific embeddings, and Manhattan or cosine distance (often more stable than Euclidean in high-d). If d is huge and you cannot reduce it, avoid KNN.

8.3 Failure 3 - Outliers (and a badly chosen K)

Picture a blue cluster that contains two stray pink points (outliers). With a small K, a new point near those pink points will be labelled pink; the decision boundary becomes pink in this pocket, blue in the next, and pink again - although that region **should have remained blue**. It became pink only because of the outliers. KNN is very sensitive to this when K is not set properly, and the model **overfits**.



Yellow rings are the outliers. K = 1 carves a pink hole around each; K = 11 simply out-votes them.

PRO TIP
Fixes: increase K (cross-validate it), remove or clip obvious outliers, use distance-weighted voting, or use a robust distance such as Manhattan.

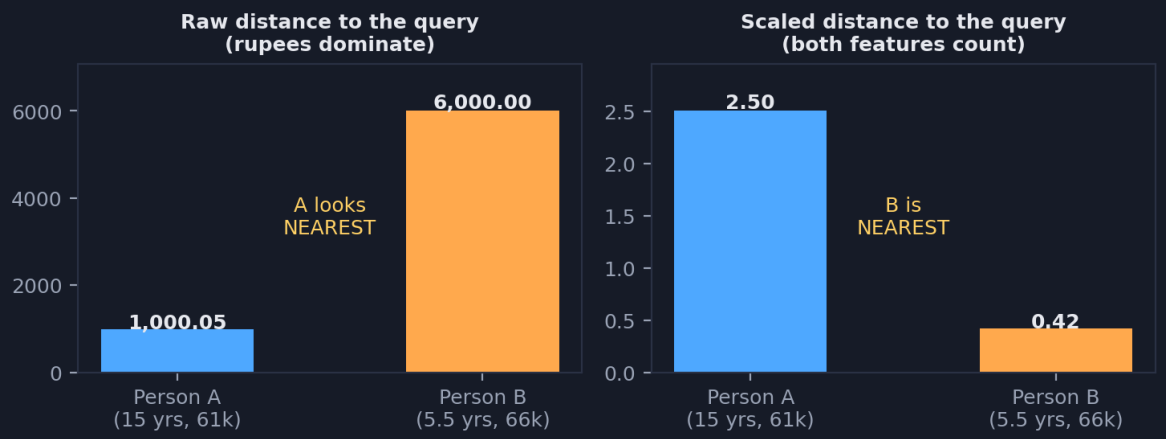
8.4 Failure 4 - Features on different scales

Suppose a person has **salary** (tens of thousands of rupees) and **experience** (a few years). The salary component will dominate the experience component, so the distance is no longer reliable - and if distance is unreliable, KNN is unreliable. That is why we said at the start: if features differ in scale you **must scale them**. Here is a number-level example:

	Experience (yrs)	Salary (Rs)	Raw distance	Scaled distance
Query	5	60,000	-	-
Person A	15	61,000	1,000.0	2.50

	Experience (yrs)	Salary (Rs)	Raw distance	Scaled distance
Person B	5.5	66,000	6,000.0	0.42

Raw: A is "nearest" (1,000 vs 6,000) even though A has **10 more years** of experience! After standardising (assuming standard deviations of 4 years and Rs 15,000): A = sqrt(2.5^2 + 0.067^2) = **2.50**, B = sqrt(0.125^2 + 0.4^2) = **0.42**. Now B is correctly the nearest.



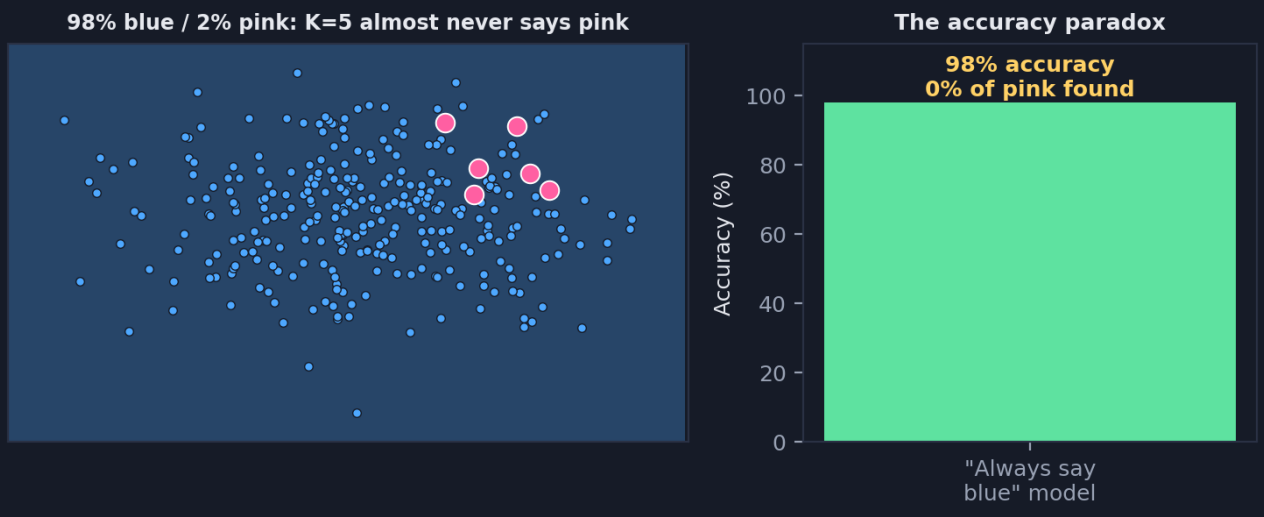
Ranking flips after scaling. Without it, salary (huge numbers) completely drowns experience.

PRO TIP

Fix: StandardScaler (z-score) or MinMaxScaler, always fit on the training data only. Put it in a Pipeline (Part 9) so you can never forget it.

8.5 Failure 5 - Imbalanced data

Suppose 98% of points are "yes" (pink in the lecture's picture, blue in ours). The data looks lopsided, and the result will be **biased towards the majority class**: among any K neighbours the majority class almost always wins the vote. If your dataset is imbalanced, KNN is one of the algorithms that will not give very good results unless you handle it.



294 blue vs 6 pink points. KNN (K=5) correctly finds 0 of the 6 pink points (it never predicts pink on its own training data here). A model that says "blue" always scores 98% accuracy - which is useless.

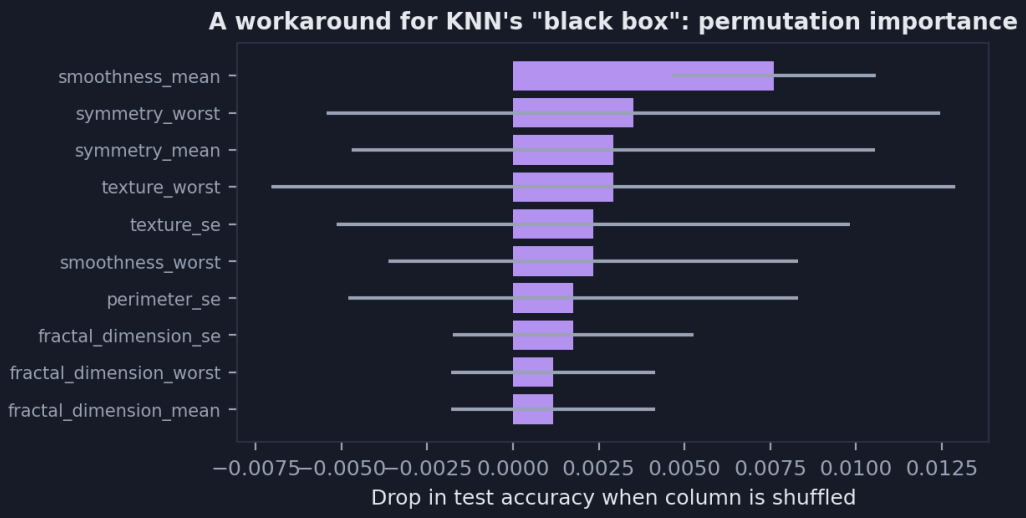
PRO TIP

Fixes: use weights="distance", resample (oversample the minority with SMOTE, or undersample the majority), use **stratified** splits, and judge the model by **recall / precision / F1 / ROC-AUC**, never by accuracy alone.

8.6 Failure 6 - Good for prediction, not for inference

KNN is good at **prediction** - telling the class of a new point. It is not good at **inference** - understanding the relationship between inputs and output. You cannot say whether CGPA contributed more or IQ contributed more to the decision. In Linear Regression the coefficients tell you how much each input affects Y; KNN has **no coefficients**. It is a **black-box model**: give it any new point and it says class 1 or class 0 - that is all.

	Logistic / Linear model	KNN
Learned parameters	A coefficient for each feature (e.g. radius_se = 1.25)	None - the data is the model
Can say "feature X matters more than Y"?	Yes	No (directly)
Prediction speed	Very fast	Slow for big N



A workaround: permutation importance shuffles one column at a time and measures the accuracy drop. On this near-perfect dataset the drops are tiny (largest is 0.8 points for smoothness_mean) because many features are correlated - treat it as a rough guide only.

8.7 Summary of the six failure cases

#	Failure case	Why it happens	Remedy
1	Large dataset	All work (N distances + sort + vote) happens at prediction time - high latency	KD/Ball-tree, ANN, sampling, PCA
2	High dimensions	Curse of dimensionality: distance loses meaning	Feature selection, PCA, Manhattan/cosine
3	Outliers, wrong K	Small K lets stray points decide - overfitting	Tune K, clean outliers, weights
4	Different scales	Big-number features dominate distance	StandardScaler / MinMaxScaler
5	Imbalanced data	Majority class wins most votes	Resample, weights, recall/F1
6	No inference	No coefficients - black box	Permutation importance, or use another model

LEVEL 4 | PRO

Part 9 - Pro-Level KNN

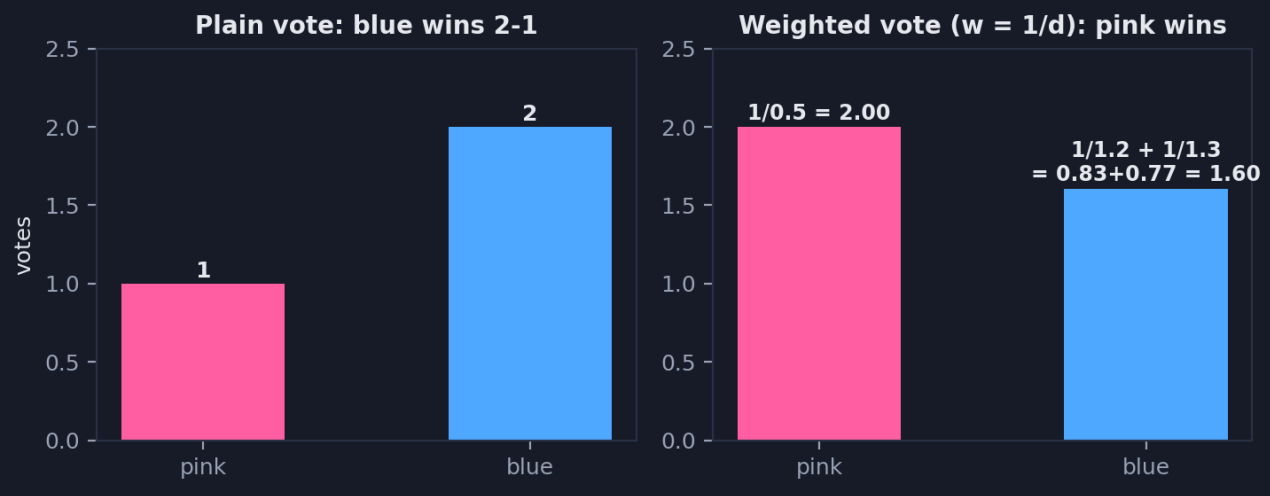
Level 4: the techniques practitioners actually use.

9.1 Weighted KNN: closer neighbours deserve a louder vote

In plain KNN, a neighbour at distance 0.5 and one at distance 1.3 each get exactly one vote. Weighted KNN gives each neighbour a weight that shrinks with distance.

DISTANCE-WEIGHTED VOTE

$$\hat{y} = \text{argmax}_c \sum_{i \in N_k(q)} w_i \mathbf{1}(y_i = c), \qquad w_i = \frac{1}{d_i}$$



Same three neighbours (distances 0.5, 1.2, 1.3). Plain vote: blue wins 2-1. Weighted vote: the very close pink neighbour (weight 2.0) beats the two far blue ones (1.60 in total).

Weighted KNN in scikit-learn

```
knn_w = KNeighborsClassifier(n_neighbors=5, weights='distance') # default is 'uniform'
# NOTE: if you hit a dtype error with string labels, encode labels as 0/1 first:
y_train_num = (y_train == 'M').astype(int)
knn_w.fit(X_train, y_train_num)
```

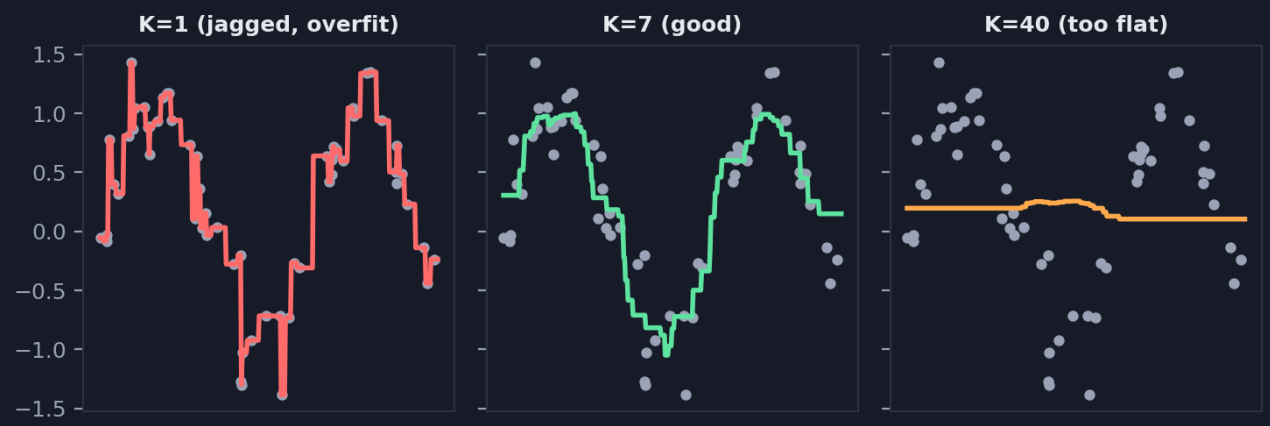
NOTE
On our breast-cancer split weighted and plain KNN both score 97.4% - weighting matters most with noisy data, imbalanced classes, or large K. It also nearly eliminates ties.

9.2 KNN for regression

Replace the vote by an **average**: predict a number by averaging the values of the K nearest neighbours (house price = average price of the 5 most similar houses).

KNN REGRESSION

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(q)} y_i \qquad \text{or weighted:} \quad \hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}$$



Same story as classification: K = 1 chases every noisy point (overfit), K = 40 flattens the curve (underfit), a middle K follows the trend.

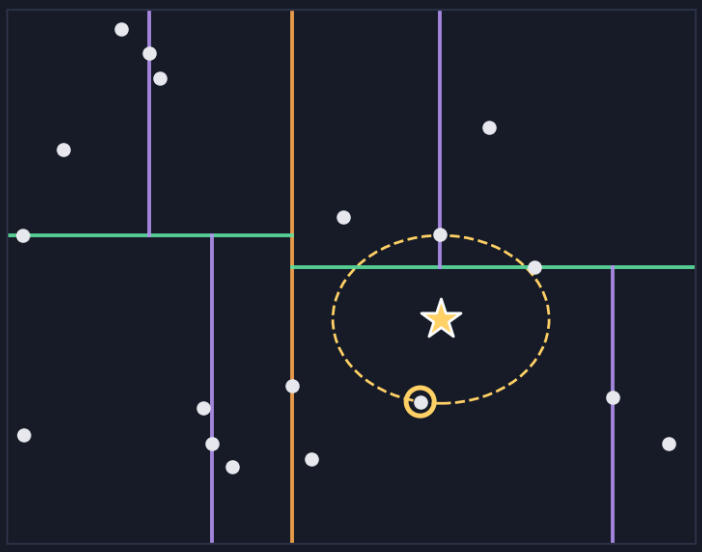
Regression

```
from sklearn.neighbors import KNeighborsRegressor
reg = KNeighborsRegressor(n_neighbors=7).fit(X_train, y_train)
print(reg.score(X_test, y_test)) # R^2, like in Linear Regression
```

9.3 Making KNN fast

Brute force compares the query with every point. Smarter structures prune most of the work.

KD-tree: space cut into boxes; far boxes skipped



A KD-tree repeatedly cuts the space along one feature at a time. If a box is farther than the best neighbour found so far, the whole box is skipped.

Method	Idea	Best when
brute	Compare with all N points	Small N, very high d, cosine/odd metrics
kd_tree	Axis-aligned space partitioning	Low dimension (roughly d below 15-20)
ball_tree	Nested hyper-spheres	Moderate d, non-Euclidean metrics
Approximate NN (FAISS, Annoy, HNSW)	Accept a tiny error for a huge speedup	Millions of points, embeddings, production search

Measured per-query time (200,000 points)	brute	kd_tree	ball_tree
d = 3 features	0.92 ms	0.014 ms	0.029 ms
d = 50 features	1.3 ms	45.7 ms	40.8 ms

WATCH OUT

Trees are about **65x faster** in 3 dimensions but about **35x slower** than brute force in 50 dimensions - the curse of dimensionality again. scikit-learn's default *algorithm='auto'* chooses sensibly; for large d use brute force or approximate search.

9.4 Fighting high dimensions

- **Feature selection**: drop irrelevant or redundant columns (irrelevant features add noise to every distance).
- **PCA** before KNN: keeps the directions with most variance.
- **Feature weighting / metric learning** (e.g. NCA): learn a distance where same-class points are close.
- Use **Manhattan** or **cosine** distance.

PCA + KNN

```
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scale', StandardScaler()),
```

```
                ('pca', PCA(n_components=10)),
                ('knn', KNeighborsClassifier(n_neighbors=7))])
```

9.5 Imbalanced data - a practical recipe

- Split with **stratify=y** so train and test keep the same class ratio.
- Use **weights="distance"** and a slightly larger K.
- Resample **only the training set**: SMOTE (imbalanced-learn) or random undersampling.
- Evaluate with **recall, precision, F1, PR-AUC**; adjust the decision threshold on predict_proba.

9.6 The production pattern: Pipeline + GridSearchCV

A **Pipeline** glues scaling and the model together, so the scaler is fitted only on the training folds inside cross-validation (no leakage) and you can never forget to scale new data.

Full pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

y_num = (y == 'M').astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y_num, test_size=0.2, random_state=2)

pipe = Pipeline([('scaler', StandardScaler()),
                 ('knn', KNeighborsClassifier())])
params = {'knn__n_neighbors': range(1, 32, 2), # odd K only
          'knn__weights': ['uniform', 'distance'],
          'knn__p': [1, 2]} # Manhattan vs Euclidean
gs = GridSearchCV(pipe, params, cv=5, scoring='f1').fit(X_train, y_train)

print(gs.best_params_) # {'knn__n_neighbors': 9, 'knn__p': 2, 'knn__weights': 'uniform'}
print(gs.best_score_) # CV F1 about 0.957
print(gs.best_estimator_.score(X_test, y_test)) # test accuracy about 0.974
```

9.7 KNN completely from scratch (NumPy)

Because KNN has no training, the from-scratch version is tiny. We checked it against scikit-learn on our split: **identical predictions** for K = 3 (99.1%) and K = 5 (97.4%).

KNN from scratch

```
import numpy as np
from collections import Counter

class MyKNN:
    def __init__(self, k=5):
        self.k = k

    def fit(self, X, y): # "training" = just store the data
        self.X = np.asarray(X, float)
        self.y = np.asarray(y)
        return self

    def _predict_one(self, q):
        d = np.sqrt(((self.X - q) ** 2).sum(axis=1)) # distance to ALL training points
        idx = np.argsort(d)[:self.k] # K nearest
        return Counter(self.y[idx]).most_common(1)[0][0] # majority vote

    def predict(self, X):
        return np.array([self._predict_one(q) for q in np.asarray(X, float)])
```

```
mine = MyKNN(k=5).fit(X_train, y_train)
print((mine.predict(X_test) == y_test).mean()) # 0.9737 - same as scikit-learn
```

9.8 Theory corner: how good can KNN get?

COVER-HART BOUND (infinite data)

$$\varepsilon_{\text{Bayes}} \leq \varepsilon_{\text{1NN}} \leq 2\,\varepsilon_{\text{Bayes}}(1-\varepsilon_{\text{Bayes}})$$

With enough data even the simple 1-nearest-neighbour rule makes at most about **twice the error of the best possible classifier** (the Bayes classifier). It is a strong theoretical reason why KNN is a respectable baseline - and why the failure cases are about *finite data, high dimension, noise and speed*, not about the idea itself.

9.9 Other uses of the neighbour idea

- **Recommender systems**: "users like you also liked ...".
- **Anomaly detection**: a point whose distance to its K-th neighbour is unusually large is an outlier.
- **Missing-value imputation**: KNNImputer fills a missing value with the average of the neighbours' values.
- **Semantic search / RAG**: embeddings + nearest-neighbour search (approximate NN).

9.10 Hyper-parameter cheat sheet

Parameter	Options	Effect
n_neighbors	1, 3, 5 ... (odd)	Main bias-variance dial
weights	'uniform', 'distance'	Closer neighbours count more
metric / p	euclidean, manhattan, cosine ... p=1,2	Changes what "close" means
algorithm	'auto','brute','kd_tree','ball_tree'	Speed only, not results
leaf_size	default 30	Speed/memory of trees
n_jobs	-1	Use all CPU cores

FINISH LINE

Part 10 - Revision Kit

Everything in one place: summary, decision guide, interview questions, practice.

10.1 One-page summary

Topic	Remember
Idea	Classify by the majority vote of the K nearest training points.
Training	None (lazy learner); all computation happens at prediction.
Distance	Euclidean by default; Manhattan, cosine, Hamming when suited.
Scaling	Mandatory. fit_transform on train, transform on test.
Choosing K	Odd number; start near sqrt(N); decide by cross-validation.
Small K	Overfitting: jagged boundary, high variance.
Large K	Underfitting: smooth to flat, ends in majority class at K = N.
Decision surface	Colour a dense grid with predictions; mlxtend does it in one line.
Failures (6)	Large data - high dimensions - outliers - different scales - imbalance - no inference.

10.2 Should I use KNN? A quick guide

Situation	KNN?
Small/medium data, few features, scaled, balanced	Great baseline
Need instant predictions on millions of rows	Avoid (or use approximate search)
Hundreds of raw features	Avoid, or reduce dimensions first
Strong class imbalance	Only with weights/resampling and proper metrics
You must explain which feature matters	Choose an interpretable model
Non-linear boundary, no assumptions wanted	Good fit

10.3 Interview questions

Question	Short answer
Why is KNN called a lazy learner?	It does no work in fit(); it stores the data and computes everything at prediction time.
What happens when K = 1? When K = N?	K = 1 overfits (memorises, high variance). K = N always predicts the majority class (underfits, high bias).
Why must features be scaled?	Distance is dominated by large-scale columns; scaling lets every feature contribute fairly.
Why fit the scaler on train only?	To avoid data leakage; the test set must mimic unseen data.
Why choose an odd K?	To avoid ties in binary classification.
How do you pick K?	Square-root heuristic as a start, then cross-validation over many K.
What is the curse of dimensionality?	In high dimensions all points are nearly equally far apart, so distances lose meaning.
How would you speed up KNN?	KD/Ball-tree in low d, approximate NN libraries, fewer points or features.
Is KNN good for inference?	No. It has no coefficients; it only predicts.
How do you handle imbalanced data with KNN?	Distance weights, resampling (SMOTE), stratified splits, evaluate with recall/F1.

10.4 Practice exercises

- Re-run the slider demo for K = 1, 3, 10, 30, 100 and write one sentence per K about the decision surface.
- Remove StandardScaler from the breast-cancer pipeline and measure the accuracy drop (we saw 97.4% to 91.2%).
- Compare Euclidean, Manhattan (p = 1) and Chebyshev. Which wins? (Manhattan scored 96.5% on our split.)
- Add 50 random noise columns and watch accuracy fall - your own curse-of-dimensionality experiment.
- Make the dataset imbalanced (keep 10 malignant cases) and compare accuracy with recall.
- Implement KNN regression from scratch by changing one line of MyKNN.
- Time brute vs kd_tree on your laptop for d = 3 and d = 50.

10.5 Glossary

Term	Definition
Lazy learner	Stores training data; computes at prediction time.
Hyper-parameter	A setting you choose (K, metric), not learned from data.
Decision boundary	The line/surface where the predicted class changes.
Overfitting	Model memorises noise; good on train, poor on new data.
Underfitting	Model too simple to capture the pattern.
Data leakage	Information from the test set sneaking into training.
Cross-validation	Repeated train/validate splits to estimate performance.
Curse of dimensionality	Distances lose meaning as features grow.
Standardisation	Rescale to mean 0 and standard deviation 1.

KEY IDEA

That is KNN. You now know the intuition, the maths, the code, how K tunes the bias-variance trade-off, how to read a decision surface, and - most importantly - the six situations where KNN will let you down.

If you remember those six points, you will be able to use KNN carefully. Happy learning!